

Basic Computer Organization and Design

Objective

This chapter presents the organization and design of a basic digital computer. Although the computer is simple compared to commercial computers, it is nevertheless encompasses enough functional capabilities to demonstrate the power of a stored program general purpose device. By going through the detailed steps of the design presented in this chapter, the student will be able to understand the inner working of digital computer.

OUTLINE

- 3.1 Instruction Codes
- 3.2 Computer Registers
- 3.3 Computer Instructions
- 3.4 Timing and Control
- 3.5 Instruction Cycle
- 3.6 Memory-Reference Instructions
- 3.7 Input-Output and Interrupt
- 3.8 Complete Computer Description
- 3.9 Design of Basic Computer
- 3.10 Design of Accumulator Logic

3.1 INSTRUCTION CODE

An instruction code is a group of bits that instruct the computer to perform a specific operation.

Operation Code

The operation code of an instruction is a group of bits that define such operations as add, subtract, multiply, shift, and complement. The number of bits required for the operation code in an instruction depends on the total number of operations available in the computer. The operation code must consist of at least n bits for a given 2 (or less) distinct operations.

Accumulator (AC)

Computers that have a single-processor register usually assign to it the name accumulator (AC) and label it AC. The operation is performed with the memory operand and the content of AC.

Stored Program Organization

The simplest way to organize a computer is to have one processor register and an instruction code format with two parts. The first part specifies the operation to be performed and the second specifies an address and the memory address tells the control where to find an operand in memory. This operand is read from memory and used as the data to be operated on together with the data stored in the processor register. The following figure 3.1 shows this type of organization.

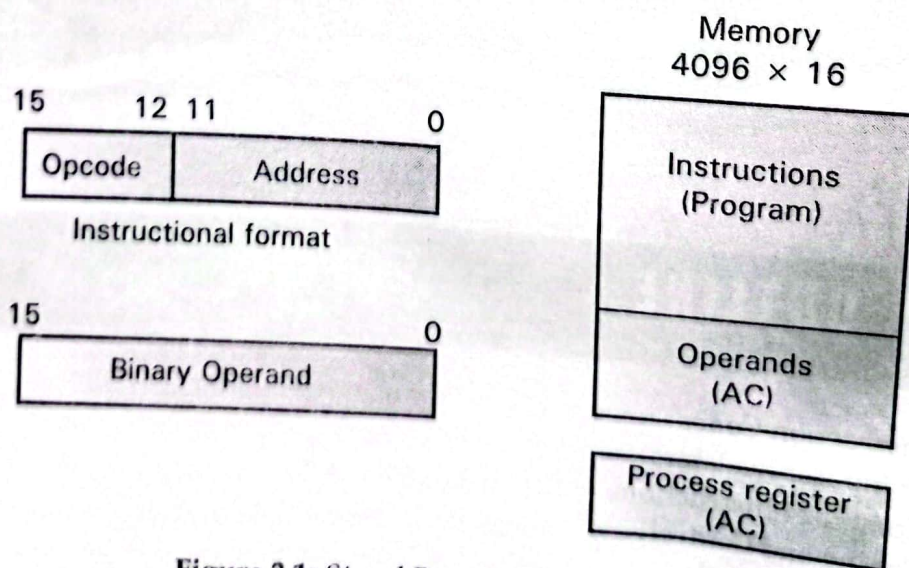


Figure 3.1: Stored Program Organization

Instructions are stored in one section of memory and data in another. For a memory unit with 4096 words, we need 12 bits to specify an address since $2^{12} = 4096$. If we store each instruction code in one 16-bit memory word, we have available four bits for operation code (abbreviated as code) to specify one out of 16 possible operations, and 12 bits to specify the address of an operand. The control reads a 16-bit instruction from the program portion of memory.

It uses the 12-bit address part of the instruction to read a 16-bit operand from the data portion of memory. It then executes the operation specified by the operation code.

Computers that have a single-processor register usually assign to it the name accumulator and label it AC. If an operation in an instruction code does not need an operand from memory, the rest of the bits in the instruction can be used for other purposes. For example, operations such as clear AC, complement AC, and increment AC operate on data stored in the AC register. They do not need an operand from memory. For these types of operations, the second part of the instruction code (bits 0 through 11) is not needed for specifying a memory address and can be used to specify other operations for the computer.

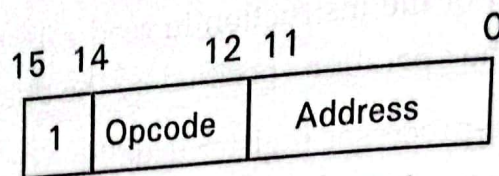
Direct and Indirect Addressing of Basic Computer

The second part of an instruction format specifies the address of an operand, the instruction is said to have a **direct address**. In **Indirect address**, the bits in the second part of the instruction designate an address of a memory word in which the address of the operand is found. One bit of the instruction code can be used to distinguish between a direct and an indirect address. It consists of a 3-bit operation code, a 12-bit address, and an indirect address mode bit designated by I where, the mode bit is 0 for a direct address and 1 for an indirect address. A direct address instruction is shown in Figure 3.2. It is placed in address 22 in memory. The I bit is 0, so the instruction is recognized as a direct address instruction. The op code specifies an ADD instruction, and the address part is the binary equivalent of 457. The control finds the operand in memory at address 457 and adds it to the content of AC.

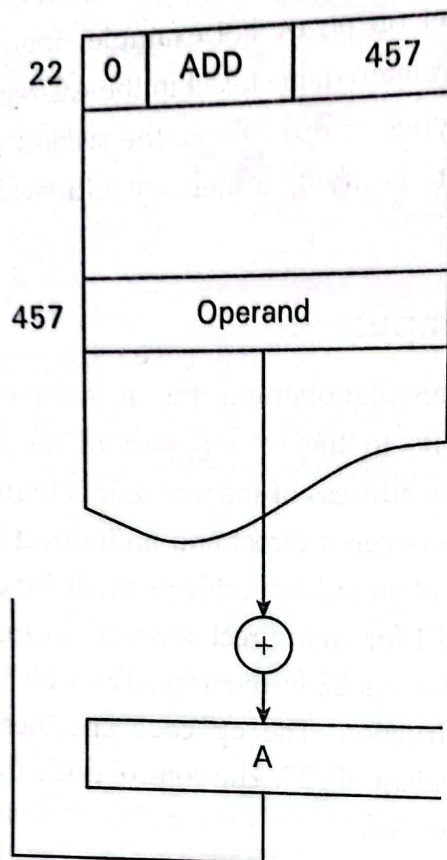
The instruction in address 35 shown in Figure 3.2 has a mode bit $I = 1$, recognized as an indirect address instruction. The address part is the binary equivalent of 300. The control goes to address 300 to find the address of the operand. The address of the operand in this case is 1350. The operand found in address 1350 is then added to the content of AC. The indirect address instruction needs two references to memory to fetch an operand.

- The first reference is needed to read the address of the operand
- Second reference is for the operand itself.

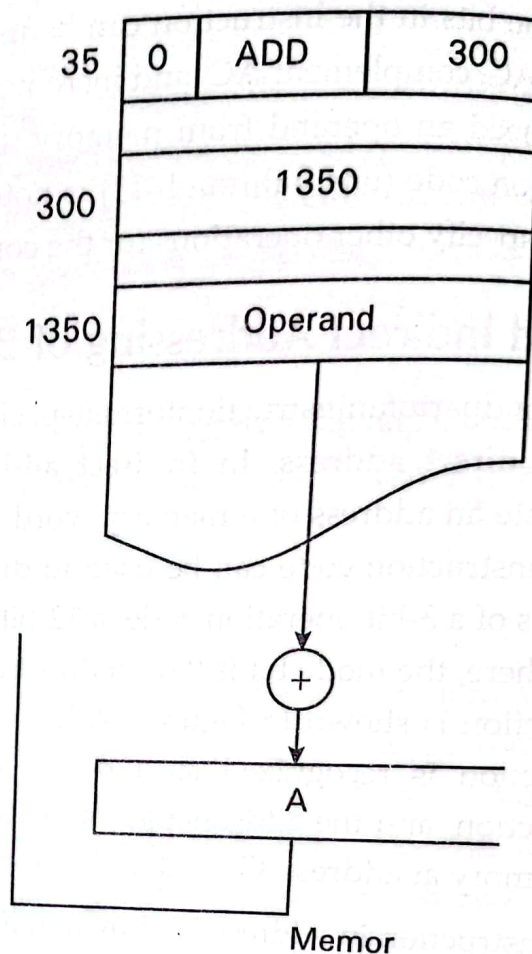
The memory word that holds the address of the operand in an indirect address instruction is used as a pointer to an array of data.



(a) Instruction format



(b) Direct address



(c) Indirect address

Figure 3.2: Direct and indirect addressing of a basic computer

Direct Address

When the second part of an instruction code specifies the address of an operand, the instruction is said to have a direct address.

For instance the instruction `MOV R0 00H`, when converted to machine language is the physical address of register R0. The instruction moves 0 to R0.

Indirect Address

When the second part of an instruction code specifies the address of a memory word in which the address of the operand, the instruction is said to have a direct address.

For instance the instruction `MOV @R0 00H`, when converted to machine language, `@R0` becomes whatever is stored in R0, and that is the address used to move 0 to. It can be whatever is stored in R0.

3.2 COMPUTER REGISTERS

It is necessary to provide a register in the control unit for storing the instruction code after it is read from memory. The computer needs processor registers for manipulating data and a

register for holding a memory address. These requirements dictate the register configuration shown in Figure 3.3.

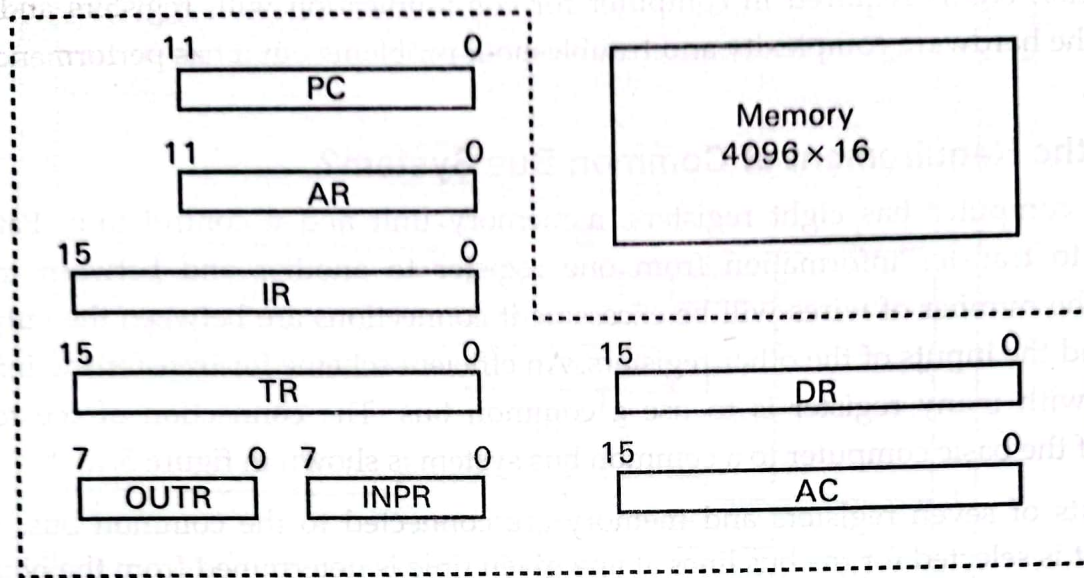


Figure 3.3: Basic Computer Register and Memory

The data register (DR) holds the operand read from memory and the accumulator (AC) register is a general purpose process in register. The instruction read from memory is placed in the instruction register (IR). The temporary register (TR) is used for holding temporary data during the processing.

The memory address register (AR) has 12 bits and the program counter (PC) also has 12 bits and it holds the address of the next instruction to be read from memory after the current instruction is executed. Instruction words are read and executed in sequence unless a branch instruction is encountered. A branch instruction calls for a transfer to a nonconsecutive instruction in the program. Two registers are used for input and output. The input register (INPR) receives an 8-bit character from an input device. The output register (OUTR) holds an 8-bit character for an output device.

Register Symbol	Bits	Register Name	Function
DR	16	Data register	Holds memory operand
AR	12	Address register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction register	Holds instruction code
PC	12	Program counter	Holds address of instruction
TR	16	Temporary register	Holds temporary data
INPR	8	Input register	Holds input character
OUTR	8	Output register	Holds output character

Table 3.1: List of Registers for Basic Computer

Common Bus System for basic computer register

The common bus is required in computer for communication with registers and memory to decrease the hardware complexity and troubleshoot problems but it has performance bottleneck limitation.

What is the Requirement of Common Bus System?

The basic computer has eight registers, a memory unit and a control unit. Paths must be provided to transfer information from one register to another and between memory and register. The number of wires will be excessive if connections are between the outputs of each register and the inputs of the other registers. An efficient scheme for transferring information in a system with many register is to use a common bus. The connection of the registers and memory of the basic computer to a common bus system is shown in figure 3.4.

The outputs of seven registers and memory are connected to the common bus. The specific output that is selected for the bus lines at any given time is determined from the binary value of the selection variables S_2 , S_1 , and S_0 . The number along each output shows the decimal equivalent of the required binary selection. The particular register whose LD (load) input is enabled receives the data from the bus during the next clock pulse transition. The memory receives the contents of the bus when its write input is activated. The memory places its 16-bit output onto the bus when the read input is activated and $S_2 S_1 S_0 = 111$.

Four registers, DR, AC, IR, and TR have 16 bits each and two registers, AR and PC, have 12 bits each since they hold a memory address. When the contents of AR or PC are applied to the 16-bit common bus, the four most significant bits are set to 0's. When AR and PC receive information from the bus, only the 12 least significant bits are transferred into the register. The input register INPR and the output register OTR have 8 bits each and communicate with the eight least significant bits in the bus. INPR is connected to provide information to the bus but OTR can only receive information from the bus.

Five registers have three control inputs: LD (load), INR (increment), and CLR (clear). Two registers have only a LD input. AR must always be used to specify a memory address; therefore memory address is connected to AR. The 16 inputs of AC come from an adder and logic circuit. This circuit has three sets of inputs.

- Set of 16-bit inputs come from the outputs of AC.
- Set of 16-bits come from the data register DR.
- Set of 8-bit inputs come from the input register INPR.

The result of an addition is transferred to AC and the end carry-out of the addition is transferred to flip-flop E (extended AC bit). The clock transition at the end of the cycle transfers the content of the bus into the designated destination register and the output of the adder and logic circuit into AC.

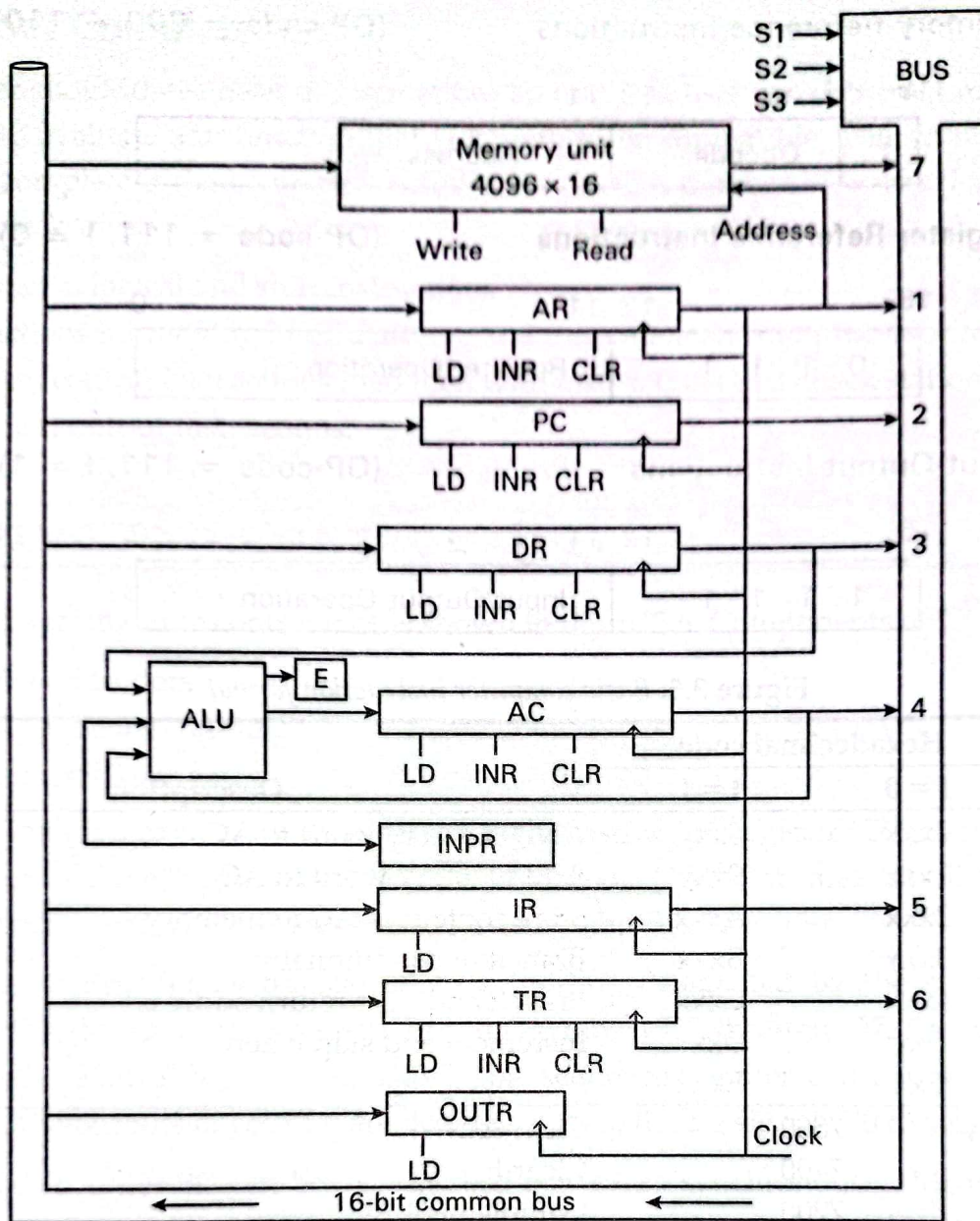


Figure 3.4: Basic computer registers connected to a common bus

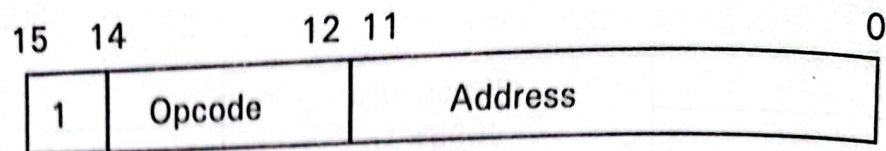
3.3 COMPUTER INSTRUCTIONS FORMAT WITH ITS TYPES

The basic computer has three instruction code formats, as shown in figure 3.5.

Each format has 16bits. The operation code (op code) part of the instruction contains three bits and the meaning of the remaining 13 bits depends on the operation code encountered. A **memory-reference instruction** uses 12 bits to specify an address and one bit to specify the addressing mode *I*. *I* is equal to 0 for direct address and to 1 for indirect address. The **register reference instructions** are recognized by the operation code 111 with a 0 in the leftmost bit (bit 15) of the instruction. A register-reference instruction specifies an operation on or a test of the AC register. An operand from memory is not needed; therefore, the other 12 bits are used to specify the operation or test to be executed. An **input-output instruction** does not need a reference to memory and is recognized by the operation code 111 with a 1 in the leftmost bit of the instruction. The remaining 12 bits are used to specify the type of input-output operation or test performed.

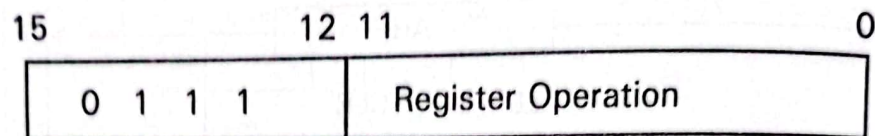
Memory-Reference Instructions

(OP-code = 000 ~ 110)



Register-Reference Instructions

(OP-code = 111, I = 0)



Input-Output Instructions

(OP-code = 111, I = 1)

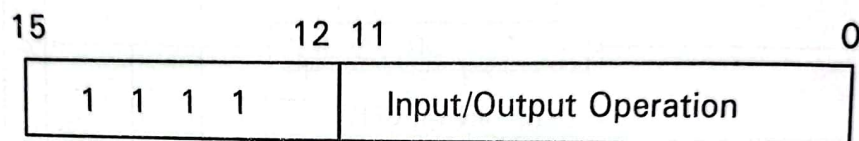


Figure 3.5: Basic computer instruction format

Symbol	Hexadecimal code		Description
	I = 0	I = 1	
AND	0xxx	8xxx	AND memory word to AC
ADD	1xxx	9xxx	Add memory word to AC
LDA	2xxx	Axxx	Store content of AC in memory
STA	3xxx	Bxxx	Branch unconditionally
BUN	4xxx	Cxxx	Branch and save return address
BSA	5xx	Dxxx	Increment and skip if zero
ISZ	6xx	Exxx	
CLA	7800		Clear AC
CLE	7400		Clear E
CMA	7200		Complement Ac
CME	7100		Complement E
CIR	7080		Circulate right AC and E
INC	7020		Circulate left AC and E
SPA	7010		Increment AC
SNA	7008		Skip next instruction if AC positive
SZA	7004		Skip next instruction if AC negative
SZE	7002		Skip next instruction if E is 0
HLT	7001		Halt computer
INP	F800		Input character to AC
OUT	F400		Output character from AC
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

Table 3.2: Basic computer instructions

Instruction Set Completeness

A computer should have a set of instructions so that that user can construct machine language programs to evaluate any function that is known to be compatible. The set of instructions are said to be complete if the computer includes a sufficient number of instructions in each of the following categories:

- Arithmetic, logical and shift instructions
- Instructions for moving information to and from memory and processor registers.
- Program control instructions along with instructions that check status conditions.
- Input and output instructions.

4 TIMING AND CONTROL UNIT

The block diagram of the control unit is shown in figure 3.6. Components of Control unit are

- ▲ Two decoders
- ▲ A sequence counter
- ▲ Control logic gates

An instruction read from memory is placed in the instruction register (IR). In control unit the IR is divided into three parts: I bit, the operation code (12-14)bit, and bits 0 through 11. The operation code in bits 12 through 14 are decoded with a 3 X 8 decoder.

Bit-15 of the instruction is transferred to a flip-flop designated by the symbol I. The eight outputs of the decoder are designated by the symbols D0 through D7. Bits 0 through 11 are applied to the control logic gates. The 4 - bit sequence counter can count in binary from 0 through 15. The outputs of counter are decoded into 16 timing signals T0 through T15.

The sequence counter SC can be incremented or cleared synchronously. Most of the time, the counter is incremented to provide the sequence of timing signals out of 4 X 16 decoder. Once in awhile, the counter is cleared to 0, causing the next timing signal to be T0. As an example, consider the case where SC is incremented to provide timing signals T0, T1, T2, T3 and T4 in sequence. At time T4, SC is cleared to 0 if decoder output D3 is active. This is expressed symbolically by the statement

$$D_3 T_4: SC \leftarrow 0$$

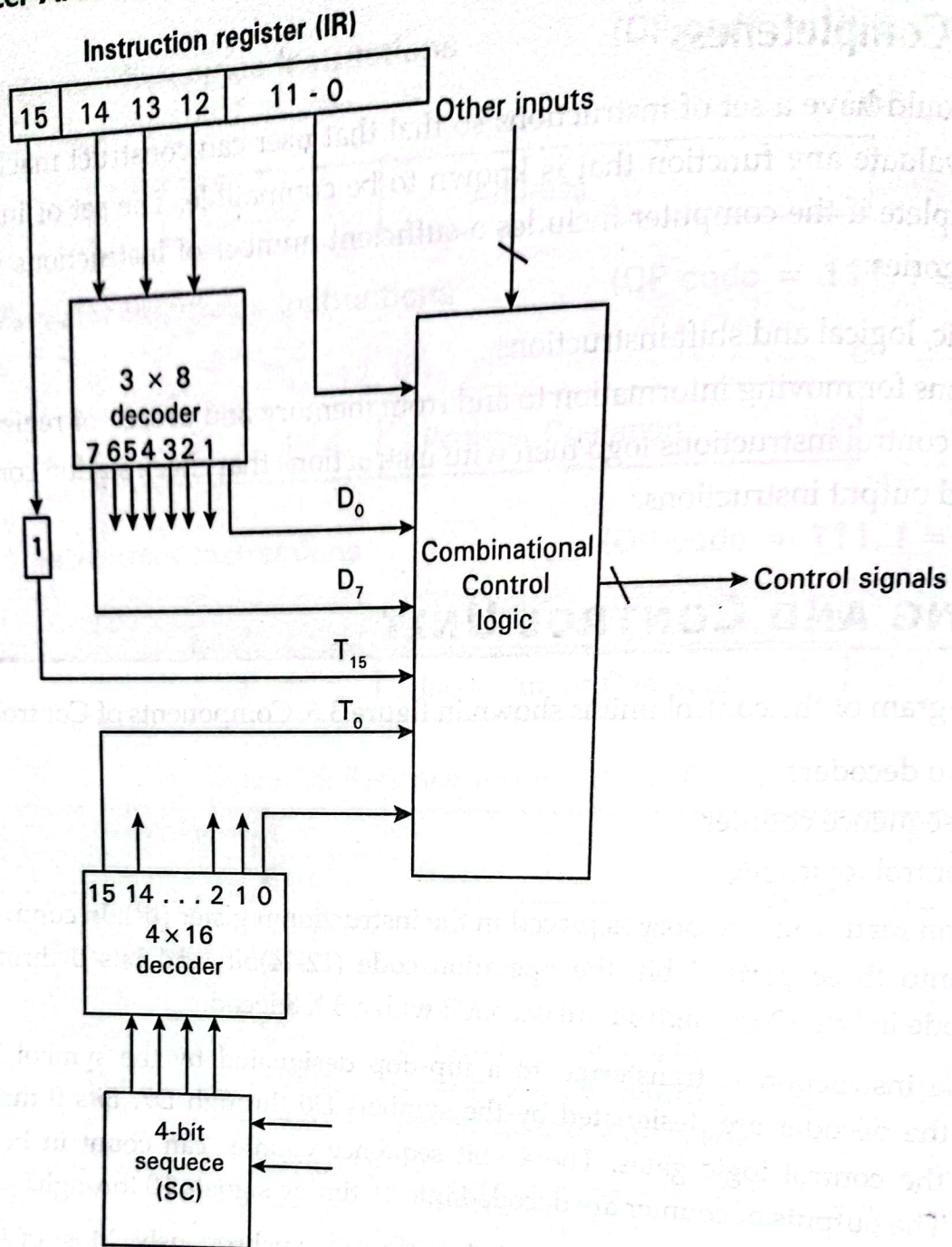


Figure 3.6: Control unit of basic computer

Timing Diagram

The timing diagram figure 3.7 shows the time relationship of the control signals. The sequence counter SC responds to the positive transition of the clock. Initially, the CLR input of SC is active. The first positive transition of the clock clears SC to 0, which in turn activates the timing signal T_0 out of the decoder. T_0 is active during one clock cycle. The positive clock transition labeled T_0 in the diagram will trigger only those registers whose control inputs are connected to timing signal T_0 . SC is incremented with every positive clock transition, unless its CLR input is active. This procedure produces the sequence of timing signals T_0, T_1, T_2, T_3 and T_4 , and so on. If SC is not cleared, the timing signals will continue with T_5, T_6 , up to T_{15} and back to T_0 .

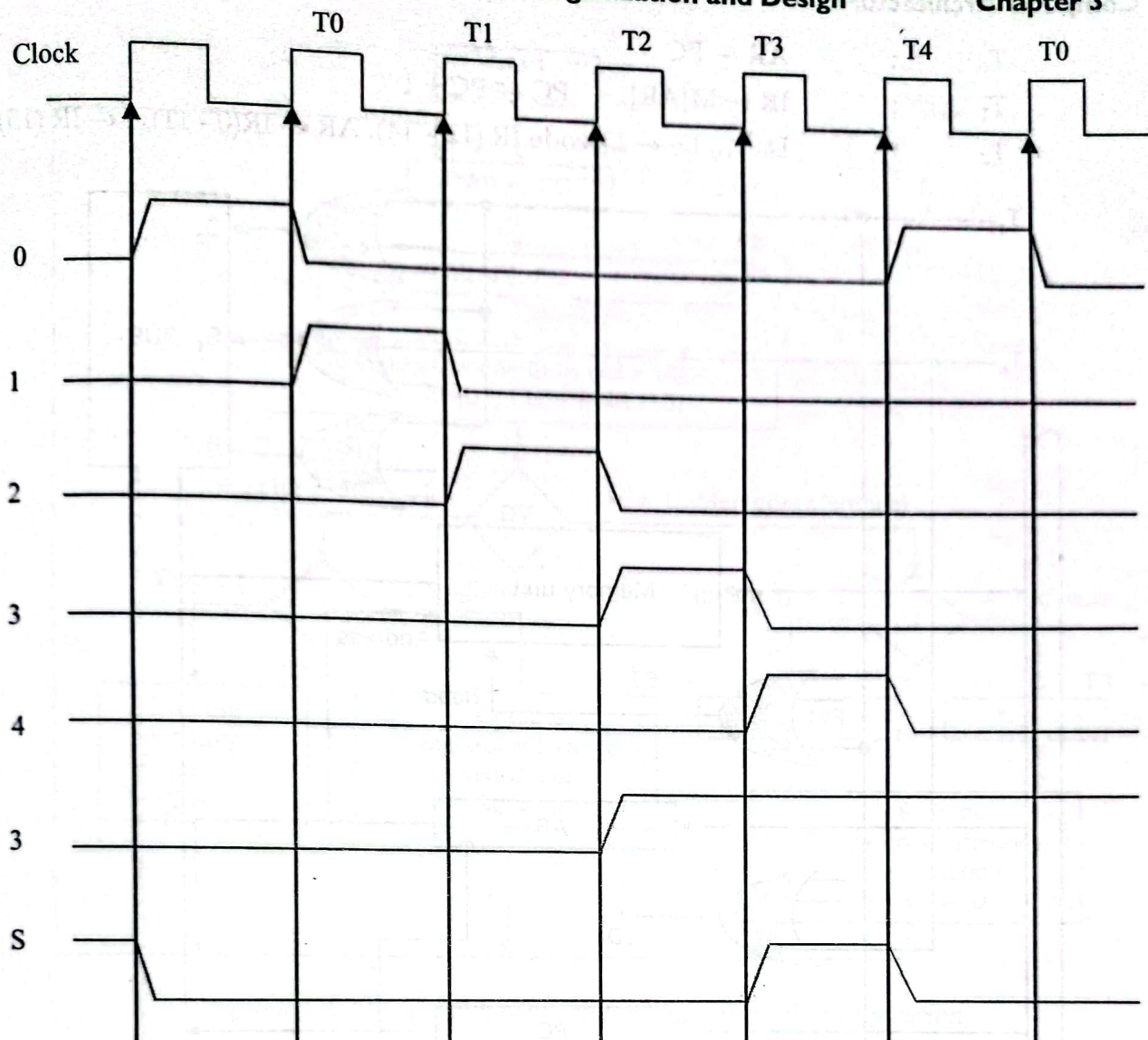


Figure 3.7: Example of control timing signals

The last three waveforms show how SC is cleared when $D_3T_4 = 1$. Output D3 from the operation decoder becomes active at the end of timing signal T_2 . When timing signal T_4 becomes active, the output of the AND gate that implements the control function D_3T_4 becomes active. This signal is applied to the CLR input of SC. On the next positive clock transition the counter is cleared to 0. This causes the timing signal T_0 to become active instead of T_5 that would have been active if SC were incremented instead of cleared.

3.5 INSTRUCTION CYCLE

- A program residing in the memory unit of the computer consists of a sequence of instructions. In the basic computer each instruction cycle consists of the following phases:
 - ▲ Fetch an instruction from memory.
 - ▲ Decode the instruction.
 - ▲ Read the effective address from memory if the instruction has an indirect address.
 - ▲ Execute the instruction.
- After step 4, the control goes back to step 1 to fetch, decode and execute the next instruction.
- This process continues unless a HALT instruction is encountered.

T_0 : $AR \leftarrow PC$
 T_1 : $IR \leftarrow M[AR], \quad PC \leftarrow PC + 1$
 T_2 : $D_0, \dots, D_7 \leftarrow \text{Decode } IR(12 - 14), AR \leftarrow IR(0 - 11), I \leftarrow IR(15)$

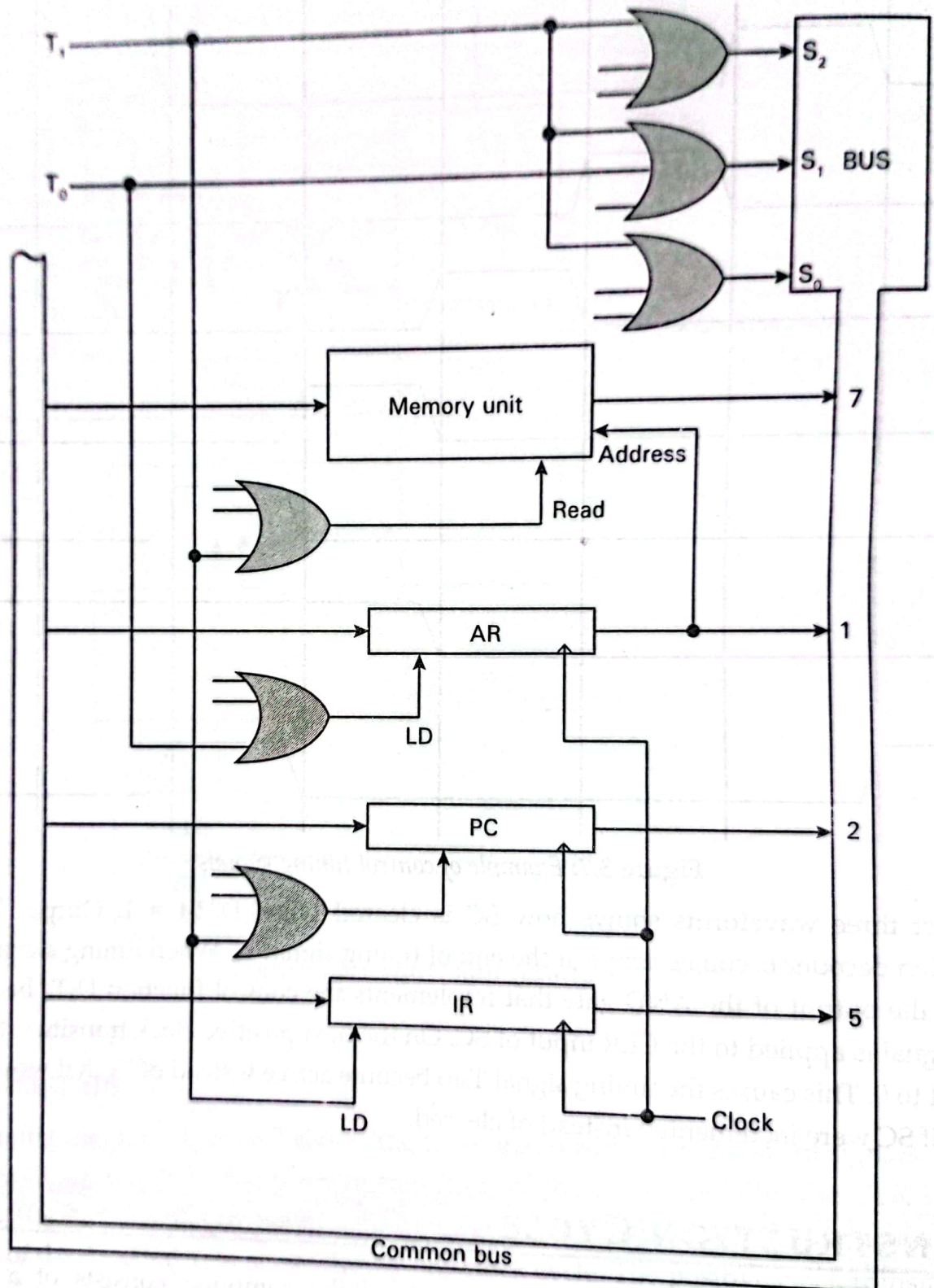


Figure 3.8: Register transfer for the fetch phase

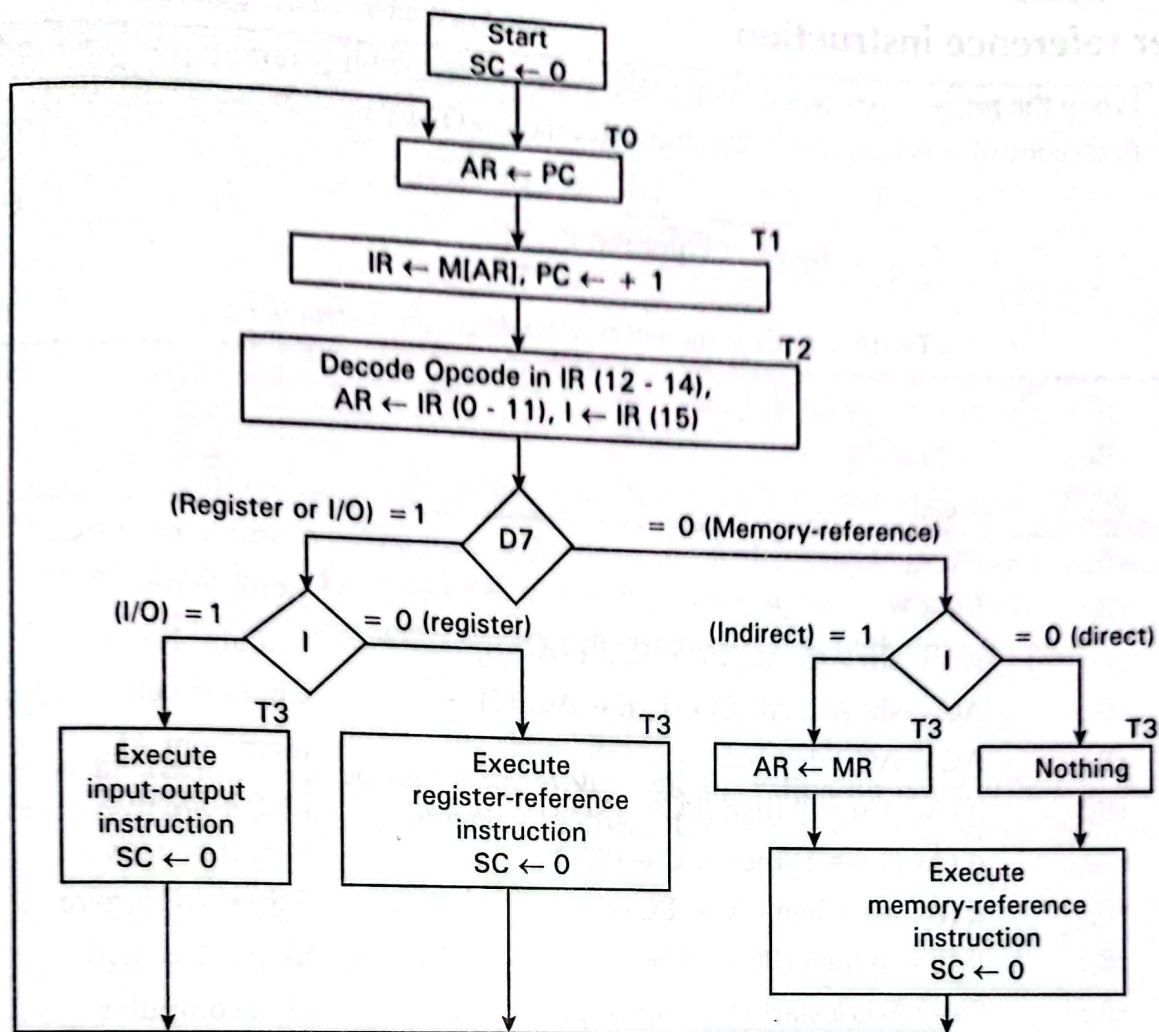


Figure 3.9: Flowchart for instruction cycle (initial configuration)

The flowchart presents an initial configuration for the instruction cycle and shows how the control determines the instruction type after the decoding. If $D7 = 1$, the instruction must be register-reference or input-output type. If $D7 = 0$, the operation code must be one of the other seven values 110, specifying a memory-reference instruction. Control then inspects the value of the first bit of the instruction, which now available in flip-flop I. If $D7 = 0$ and $I = 1$, we have a memory-reference instruction with an indirect address. It is then necessary to read the effective address from memory.

The three instruction types are subdivided into four separate paths. The selected operation is activated with the clock transition associated with timing signal T_3 . This can be symbolized as follows:

$D'7 I T_3 :$	$AR \leftarrow M[AR]$
$D7 I' T_3 :$	Execute a register-reference instruction
$D7 I T_3 :$	Execute an input-output instruction

When a memory-reference instruction with $I = 0$ is encountered, it is not necessary to do anything since the effective address is already in AR. However, the sequence counter SC must be incremented when $D'7 I T_3 = 1$, so that the execution of the memory-reference instruction can be continued with timing variable T_4 . A register-reference or input-output instruction can be executed with the clock associated with timing signal T_3 . After the instruction is executed, SC is either incremented or cleared to 0 and control returns to the fetch phase with $T_0 = 1$. SC is either incremented or cleared to 0 with every positive clock transition.

Register reference instruction

- When the register-reference instruction is decoded, D7 bit is set to 1.
- Each control function needs the Boolean relation D7 I'T3

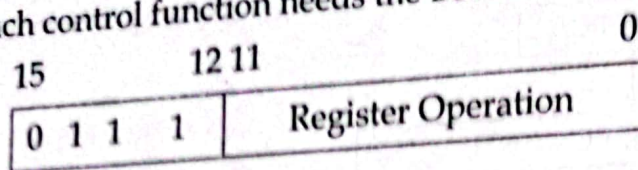


Table 3.3 : Execution of register-reference instructions

	r:	SC ← 0	Clear SC
CLA	rB ₁₁ :	AC ← 0	Clear AC
CLE	rB ₁₀ :	E ← 0	Clear E
CMA	rB ₉ :	AC ← AC'	Complement AC
CME	rB ₈ :	E ← E'	Complement E
CIR	rB ₇ :	AC ← shr AC, AC(15) ← E, E ← AC(0)	Circular Right
CIL	rB ₆ :	AC ← shl AC, AC(0) ← E, E ← AC(15)	Circular Left
INC	rB ₅ :	AC ← AC + 1	Increment AC
SPA	rB ₄ :	if (AC(15) = 0) then (PC ← PC+1)	Skip if positive
SNA	rB ₃ :	if (AC(15) = 1) then (PC ← PC+1)	Skip if negative
SZA	rB ₂ :	if (AC = 0) then (PC ← PC+1)	Skip if AC is zero
SZE	rB ₁ :	if (E = 0) then (PC ← PC+1)	Skip if E is zero
HLT	rB ₀ :	S ← 0 (S is a start-stop flip-flop)	Halt computer

- These 12 bits are available in IR (0-11). They were also transferred to AR during time T₂.
- These instructions are executed at timing cycle T₃.
- The first seven register-reference instructions perform clear, complement, circular shift and increment microoperations on the AC or E registers.
- The next four instructions cause a skip of the next instruction in sequence when condition is satisfied. The skipping of the instruction is achieved by incrementing PC.
- The condition control statements must be recognized as part of the control conditions. The AC is positive when the sign bit in AC(15) = 0; it is negative when AC(15) = 1. The content of AC is zero (AC = 0) if all the flip-flops of the register are zero.
- The HLT instruction clears a start-stop flip-flop S and stops the sequence counter from counting. To restore the operation of the computer, the start-stop flip-flop must be set manually.

3.6 MEMORY REFERENCE INSTRUCTIONS

- When the memory-reference instruction is decoded, D7 bit is set to 0.

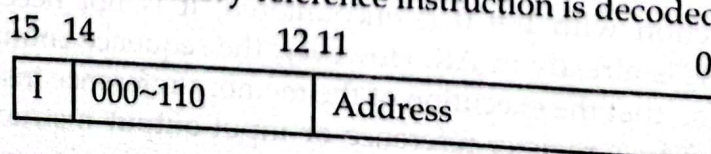


Table 3.4: Memory-reference instructions

Symbol	Operation Decoder	Symbolic Description
AND	D ₀	$AC \leftarrow AC \wedge M[AR]$
ADD	D ₁	$AC \leftarrow AC + M[AR], E \leftarrow C_{out}$
LDA	D ₂	$AC \leftarrow M[AR]$
STA	D ₃	$M[AR] \leftarrow AC$
BUN	D ₄	$PC \leftarrow AR$
BSA	D ₅	$M[AR] \leftarrow PC, PC \leftarrow AR + 1$
ISZ	D ₆	$M[AR] \leftarrow M[AR] + 1, \text{ if } M[AR] + 1 = 0 \text{ then } PC \leftarrow PC + 1$

- The effective address of the instruction is in the address register AR and was placed there during timing signal T₂ when I = 0, or during timing signal T₃ when I = 1.
- The execution of the memory-reference instructions starts with timing signal T₄.

AND to AC

This is an instruction that performs the AND logic operation on pairs of bits in AC and the memory word specified by the effective address. The result of the operation is transferred to AC.

D₀T₄: DR ← M[AR]

D₀T₅: AC ← AC ∧ DR, SC ← 0

ADD to AC

This instruction adds the content of the memory word specified by the effective address to the value of AC. The sum is transferred into AC and the output carry C_{out} is transferred to the E (extended accumulator) flip-flop.

D₁T₄: DR ← M[AR]

D₁T₅: AC ← AC + DR, E ← C_{out}, SC ← 0

LDA: Load to AC

This instruction transfers the memory word specified by the effective address to AC.

D₂T₄: DR ← M[AR]

D₂T₅: AC ← DR, SC ← 0

STA: Store AC

This instruction stores the content of AC into the memory word specified by the effective address.

D₃T₄: M[AR] ← AC, SC ← 0

BUN: Branch Unconditionally

This instruction transfers the program to instruction specified by the effective address. The BUN instruction allows the programmer to specify an instruction out of sequence and the program branches (or jumps) unconditionally.

D₄T₄: PC ← AR, SC ← 0

BSA: Branch and Save Return Address

This instruction is useful for branching to a portion of the program called a subroutine or procedure. When executed, the BSA instruction stores the address of the next instruction in sequence (which is available in PC) into a memory location specified by the effective address.

$$M[AR] \leftarrow PC, PC \leftarrow AR + 1$$

$$M[135] \leftarrow 21, PC \leftarrow 135 + 1 = 136$$

Memory, PC, AR at time T4

20	0	BSA	135
PC = 21		Next instruction	
AR = 135			
136		Subroutine	
		↓	
1		BUN	135

Memory, PC after execution

20	0	BSA	135
21		Next instruction	
AR = 135			
136		Subroutine	
		↓	
1		BUN	135

Figure 3.10: Example of BSA instruction execution

It is not possible to perform the operation of the BSA instruction in one clock cycle when we use the bus system of the basic computer. To use the memory and the bus properly, the BSA instruction must be executed with a sequence of two microoperations:

$$D_5T_4: M[AR] \leftarrow PC, AR \leftarrow AR + 1$$

$$D_5T_5: PC \leftarrow AR, SC \leftarrow 0$$

ISZ: Increment and Skip if Zero

These instruction increments the word specified by the effective address, and if the incremented value is equal to 0, PC is incremented by 1. Since it is not possible to increment a word inside the memory, it is necessary to read the word into DR, increment DR, and store the word back into memory.

$$D_6T_4: DR \leftarrow M[AR]$$

$$D_6T_5: DR \leftarrow DR + 1$$

$$D_6T_6: M[AR] \leftarrow DR, \text{ if } (DR = 0) \text{ then } (PC \leftarrow PC + 1), SC \leftarrow 0$$

Control Flowchart

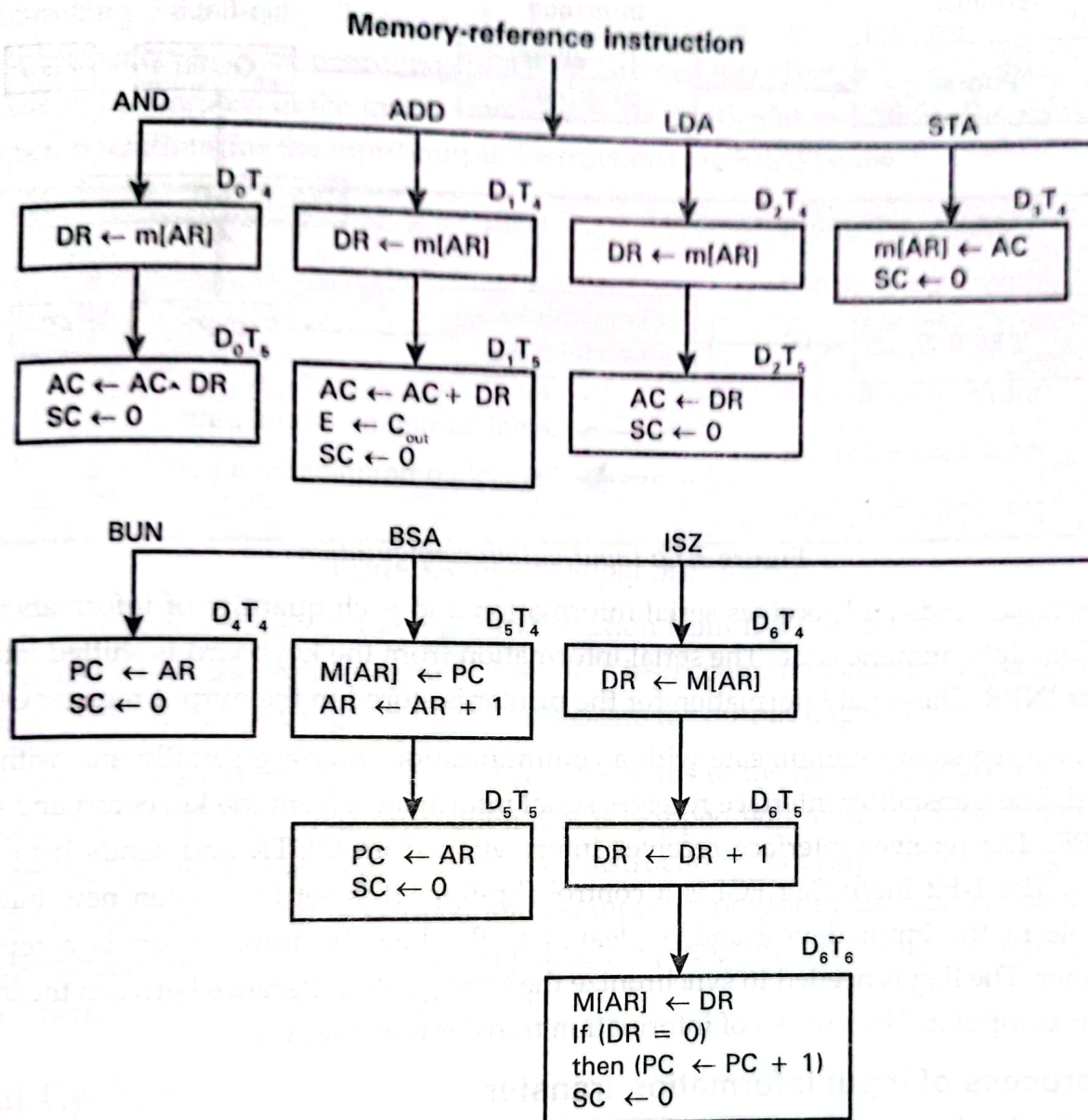


Figure 3.11: Flowchart for memory-reference instructions

3.7 INPUT-OUTPUT CONFIGURATION OF BASIC COMPUTER

A computer can serve no useful purpose unless it communicates with the external environment. To exhibit the most basic requirements for input and output communication, we will use a terminal unit with a keyboard and printer.

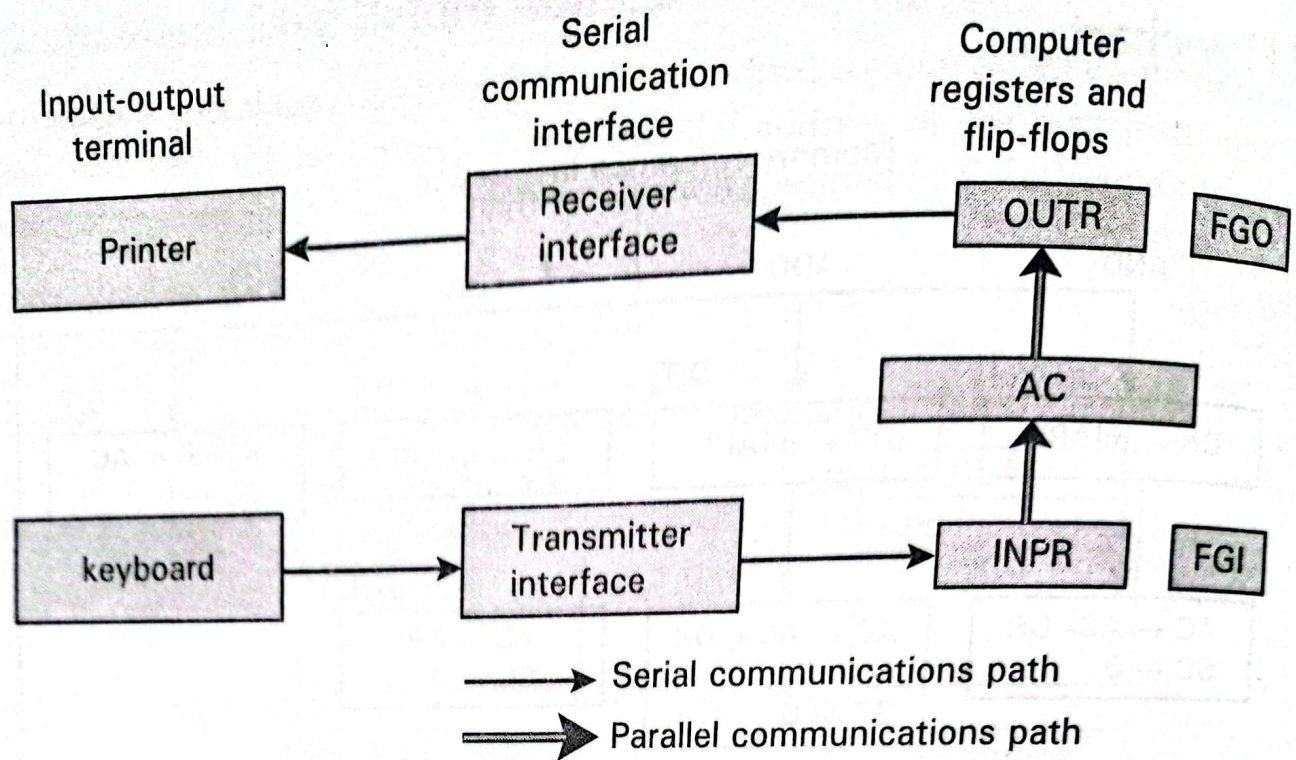


Figure 3.12: Input-output configuration

The terminal sends and receives serial information and each quantity of information has eight bits of an alphanumeric code. The serial information from the keyboard is shifted into the input register INPR. The serial information for the printer is stored in the output register OUTR.

These two registers communicate with a communication interface serially and with the AC in parallel. The transmitter interface receives serial information from the keyboard and transmits it to INPR. The receiver interface receives information from OUTR and sends it to the printer serially. The 1-bit input flag FGI is a control flip-flop. It is set to 1 when new information is available in the input device and is cleared to 0 when the information is accepted by the computer. The flag is needed to synchronize the timing rate difference between the input device and the computer. The process of information transfer is as follows:

The process of input information transfer

- Initially, the input flag FGI is cleared to 0. When a key is struck in the keyboard, an 8-bit alphanumeric code is shifted into INPR and the input flag FGI is set to 1.
- As long as the flag is set, the information in INPR cannot be changed by striking another key. The computer checks the flag bit; if it is 1, the information from INPR is transferred in parallel into AC and FGI is cleared to 0.
- Once the flag is cleared, new information can be shifted into INPR by striking another key.

The process of outputting information

- The output register OUTR works similarly but the direction of information flow is reversed.
- Initially, the output flag FGO is set to 1. The computer checks the flag bit; if it is 1, the information from AC is transferred in parallel to OUTR and FGO is cleared to 0. The output device accepts the coded information, prints the corresponding character, and when the operation is completed, it sets FGO to 1.
- The computer does not load a new character into OUTR when FGO is 0 because this condition indicates that the output device is in the process of printing the character.

Input-Output instructions

Input and output instructions are needed for transferring information to and from AC register, for checking the flag bits, and for controlling the interrupt facility. Input-output instructions have an operation code 1111 and are recognized by the control when $D7 = 1$ and $I = 1$. The remaining bits of the instruction specify the particular operation. The control functions and microoperations for the input-output instructions are listed below.

INP	$AC(0-7) \leftarrow INPR, FGI \leftarrow 0$	Input char. to AC
OUT	$OUTR \leftarrow AC(0-7), FGO \leftarrow 0$	Output char. from AC
SKI	if($FGI = 1$) then ($PC \leftarrow PC + 1$)	Skip on input flag
SKO	if($FGO = 1$) then ($PC \leftarrow PC + 1$)	Skip on output flag
ION	$IEN \leftarrow 1$	Interrupt enable on
IOF	$IEN \leftarrow 0$	Interrupt enable off

Table 3.5: *Input Output Instructions*

The INP instruction transfers the input information from INPR into the eight low-order bits of AC and also clears the input flag to 0. The OUT instruction transfers the eight least significant bits of AC into the output register OUTR and clears the output flag to 0.

The next two instructions in Table 3.5 check the status of the flags and cause a skip of the next instruction if the flag is 1. The instruction that is skipped will normally be a branch instruction to return and check the flag again. The branch instruction is not skipped if the flag is 0. If the flag is 1, the branch instruction is skipped and an input or output instruction is executed. The last two instructions set and clear an interrupt enable flip-flop IEN. The purpose of IEN is explained in conjunction with the interrupt operation.

Interrupt Cycle

The way that the interrupt is handled by the computer can be explained by means of the flowchart shown in figure 3.13. An interrupt flip-flop R is included in the computer. When $R = 0$, the computer goes through an instruction cycle. During the execute phase of the instruction cycle IEN is checked by the control. If it is 0, it indicates that the programmer does not want to use the interrupt, so control continues with the next instruction cycle. If IEN is 1, control checks the flag bits. If both flags are 0, it indicates that neither the input nor the output registers are ready for transfer of information. In this case, control continues with the next instruction cycle. If either flag is set to 1 while $IEN = 1$, flip-flop R is set to 1. At the end of the execute phase, control checks the value of R, and if it is equal to 1, it goes to an interrupt cycle instead of an instruction cycle.

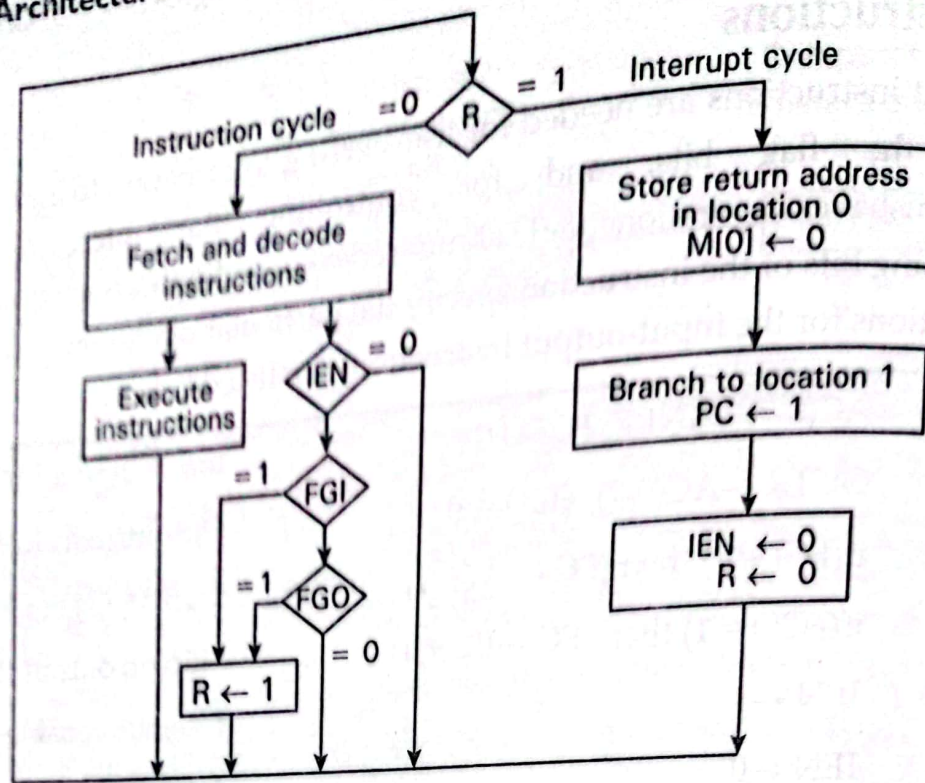


Figure 3.13: Flowchart for interrupt cycle

The interrupt cycle is a hardware implementation of a branch and save return address operation. The return address available in PC is stored in a specific location where it can be found later when the program returns to the instruction at which it was interrupted. This location may be a processor register, a memory stack, or a specific memory location. Here we choose the memory location at address 0 as the place for storing the return address. Control then inserts address 1 into PC and clears IEN and R so that no more interruptions can occur until the interrupt request from the flag has been serviced. An example that shows what happens during the interrupt cycle is shown in Figure 3.14:

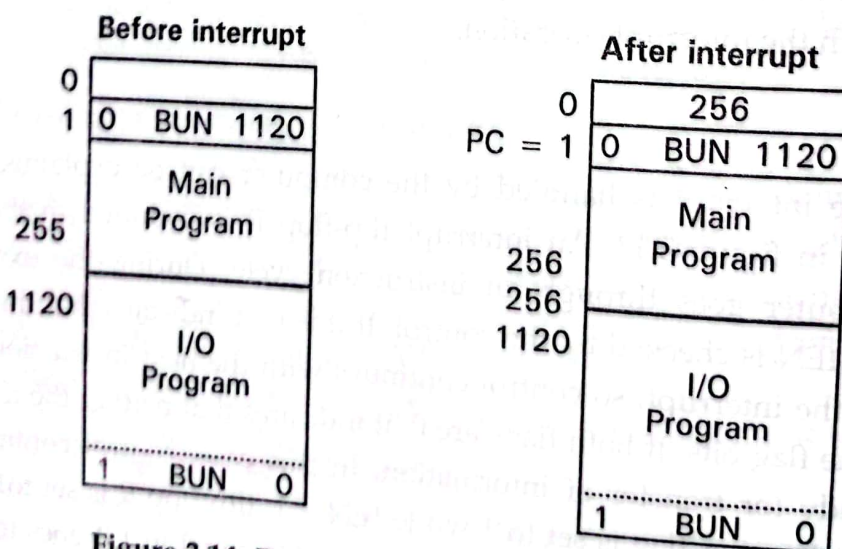


Figure 3.14: Demonstration of the interrupt cycle

Suppose that an interrupt occurs and $R=1$, while the control is executing the instruction at address 255. At this time, the return address 256 is in PC. The programmer has previously placed an input-output service program in memory starting from address 1120 and a BUN 1120 instruction at address 1. The content of PC (256) is stored in memory location 0, PC is set to 1, and R is cleared to 0. At the beginning of the next instruction cycle, the instruction that is read from memory is in address 1 since this is the content of the instruction at address 1, which causes the program to transfer control to address 0.

This program checks the flags, determines which flag is set, and then transfers the required input or output information. Once this is done, the instruction ION is executed to set IEN to 1 (to enable further interrupts), and the program returns to the location where it was interrupted. The instruction that returns the computer to the original place in the main program is a branch indirect instruction with an address part of 0. This instruction is placed at the end of the I/O service program. The execution of the indirect BUN instruction results in placing into PC the return address from location 0.

Register transfer statements for the interrupt cycle

The flip-flop is set to 1 if $IEN = 1$ and either FGI or FGO are equal to 1. This can happen with any clock transition except when timing signals T_0 , T_1 or T_2 are active. The condition for setting flip-flop $R = 1$ can be expressed with the following register transfer statement:

$$T_0'T_1'T_2'(IEN)(FGI + FGO): R \leftarrow 1$$

The symbol $+$ between FGI and FGO in the control function designates a logic OR operation. This is AND with IEN and $T_0'T_1'T_2'$. The fetch and decode phases of the instruction cycle must be modified and Replace T_0 , T_1 , T_2 with $R'T_0$, $R'T_1$, $R'T_2$. Therefore the interrupt cycle statements are:

$$RT_0: \quad AR \leftarrow 0, TR \leftarrow PC$$

$$RT_1: \quad M[AR] \leftarrow TR, PC \leftarrow 0$$

$$RT_2: \quad PC \leftarrow PC + 1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$$

During the first timing signal AR is cleared to 0, and the content of PC is transferred to the temporary register TR. With the second timing signal, the return address is stored in memory at location 0 and PC is cleared to 0. The third timing signal increments PC to 1, clears IEN and R, and control goes back to T_0 by clearing SC to 0. The beginning of the next instruction cycle has the condition RT_0 and the content of PC is equal to 1. The control then goes through an instruction cycle that fetches and executes the BUN instruction in location 1.

3.8 COMPLETE COMPUTER DESCRIPTION

Flow chart for computer operation

The final flowchart of the instruction cycle, including the interrupt cycle for the basic computer, is shown in Figure 3.15. The interrupt flip-flop R may be set at any time during the indirect or execute phases. The control returns to timing signal T_0 after SC is cleared to 0.

If $R = 1$, the computer goes through an interrupt cycle. If $R = 0$, the computer goes through an instruction cycle. If the instruction is one of the memory-reference instructions, the computer first checks if there is an indirect address and then continues to execute the decoded instruction according to the flow chart. If the instruction is one of the register-reference instructions, it is executed with one of the microoperations register reference. If it is an input-output instruction, it is executed with one of the microoperation's input-output reference.

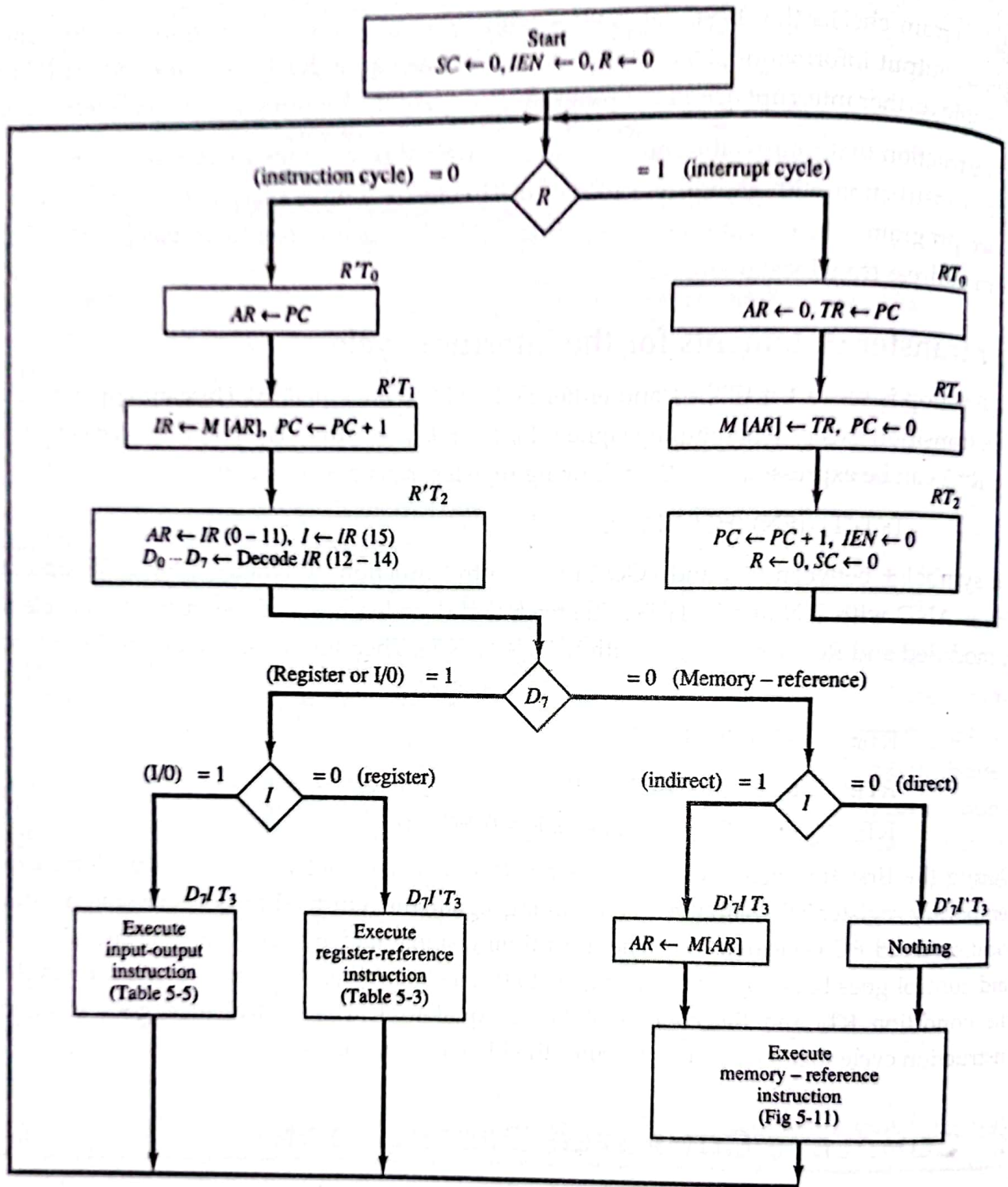


Figure 3.15: Flowchart for computer operation

Fetch

F^*T_0 : $AR \leftarrow PC$

Decode

R^*T_1 : $IR \leftarrow M[AR]$, $PC \leftarrow PC + 1$

R^*T_2 : $D_0, \dots, \leftarrow \text{Decode } IR (12-14)$,
 $AR \leftarrow IR (0-11)$, $I \leftarrow IR (15)$

Indirect interrupt:

$D_7I^*T_3$: $AR \leftarrow M[AR]$

$T_0^*T_1^*T_2^*(IEN) (FGI + FGO)$: $R \leftarrow 1$

RT_0 : $AR \leftarrow 0$, $TR \leftarrow PC$

RT_1 : $M[AR] \leftarrow TR$, $PC \leftarrow 0$

RT_2 : $PC \leftarrow PC + 1$, $IEN \leftarrow 0$, $R \leftarrow 0$, $SC \leftarrow 0$

Memory Reference:

AND

D_0T_4 : $DR \leftarrow M[AR]$

D_0T_5 : $AC \leftarrow AC \wedge DR$, $SC \leftarrow 0$

ADD	$D_1T_4: DR \leftarrow M[AR]$
LDA	$D_1T_5: AC \leftarrow AC + DR, E \leftarrow C_{out}, SC \leftarrow$
STA	$D_2T_4: DR \leftarrow M[AR]$
BUN	$D_2T_5: AC \leftarrow DR, SC \leftarrow 0$
BSA	$D_3T_5: M[AAR] \leftarrow AAC, SC \leftarrow 0$
ISZ	$D_4T_4: PC \leftarrow AR, SC \leftarrow 0$
	$D_5T_4: M[AR] \leftarrow PC, AR \leftarrow AR + 1$
	$D_5T_5: PC \leftarrow AR, SC \leftarrow 0$
	$D_6T_4: DR \leftarrow M[AR]$
	$D_6T_5: DR \leftarrow DR + 1$
	$D_6T_6: M[AR] \leftarrow DR, \text{ If } (DR = 0) \text{ then } (PC \leftarrow PC + 1), SC \leftarrow 0$
Register Reference:	
	$D_7I^*T_3 = r \text{ (common to all register reference instruction)}$
	$IR(i) = B_i \text{ (} i = 0, 1, 2, \dots, 11 \text{)}$
	$r: SC \leftarrow 0$
CLA	$rB_{11}: AC \leftarrow 0$
CLE	$rB_{10}: E \leftarrow 0$
CMA	$rB_9: AC \leftarrow \overline{AC}$
CME	$rB_8: E \leftarrow \overline{E}$
CI	$rB_7: AC \leftarrow shr AC, AC(15) \leftarrow E, E \leftarrow AC(0)$
CIL	$rB_6: AC \leftarrow shl AC, AC(0) \leftarrow E, E \leftarrow AC(15)$
INC	$rB_5: AC \leftarrow AC + 1$
SPA	$rB_4: \text{ If } (AC(15) = 0) \text{ then } (PC \leftarrow PC + 1)$
SN	$rB_3: \text{ If } (AC(15) = 1) \text{ then } (PC \leftarrow PC + 1)$
SZA	$rB_2: \text{ If } (AC = 0) \text{ then } (PC \leftarrow PC + 1)$
SZE	$rB_1: \text{ If } (E = 0) \text{ then } (PC \leftarrow PC + 1)$
HLT	$rB_0: S \leftarrow 0$
Input-Output:	
	$D_7I^*T_3 = p \text{ (common to all input-output instruction)}$
	$IR(i) = B_i \text{ (} i = 6, 7, 8, \dots, 11 \text{)}$
	$r: SC \leftarrow 0$
INP	$pB_{11}: AC(0-7) \leftarrow INPR, FGI \leftarrow 0$
OUT	$pB_{10}: OUTR \leftarrow AC(0-7), FGO \leftarrow 0$
SKI	$pB_9: \text{ If } (FGI = 1) \text{ then } (PC \leftarrow PC + 1)$
SKO	$pB_8: \text{ If } (FGO = 1) \text{ then } (PC \leftarrow PC + 1)$
ION	$pB_7: IEN \leftarrow 1$
IOF	$pB_6: IEN \leftarrow 0$

Table 3.6: Control functions and microoperations for the basic computer

3.9 DESIGN OF BASIC COMPUTER

Hardware Components of BC

A memory unit: 4096 x 16.

Registers: AR, PC, DR, AC, IR, TR, OUTR, INPR, and SC

Flip-Flops(Status): I, S, E, R, IEN, FGI, and FGO

Decoders: a 3x8 Opcode decoder

a 4x16 timing decoder

Common bus: 16 bits

Control logic gates: Adder and Logic circuit connected to AC

Control Logic Gates

- Input Controls of the nine registers
- Read and Write Controls of memory
- Set, Clear, or Complement Controls of the flip-flops
- S₂, S₁, S₀ Controls to select a register for the bus
- AC, and Adder and Logic circuit

Control of AR register

Scan all of the register transfer statements that change the content of AR:

R'T₀: AR ← PC LD(AR)

R'T₂: AR ← IR(0-11) LD(AR)

D'₇IT₃: AR ← M[AR] LD(AR)

RT₀: AR ← 0 CLR(AR)

D₅T₄: AR ← AR + 1 INR(AR)

Now,

LD(AR) = R'T₀ + R'T₂ + D'₇IT₃

CLR(AR) = RT₀

INR(AR) = D₅T₄

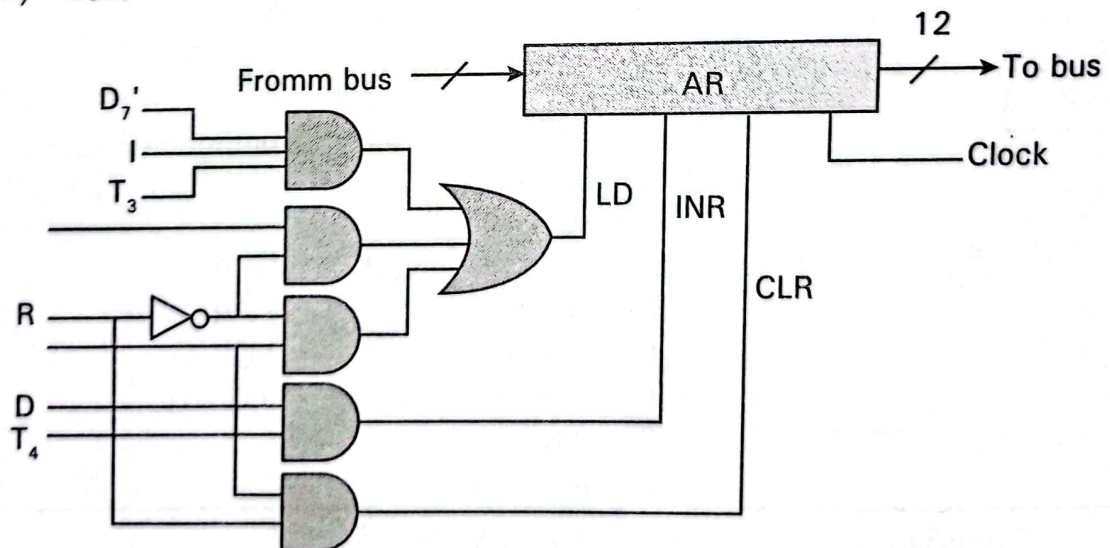


Figure 3.16: Control Gates Associated with AR

Control of IEN Flip-Flop

pB₇: IEN ← 1 (I/O Instruction)

pB₆: IEN ← 0 (I/O Instruction)

RT₂: IEN ← 0 (Interrupt)

=>

Set (IEN) = pB₇

Clear(IEN) = pB₆ + RT₂

$p = D_7IT_3$ (Input/Output Instruction)

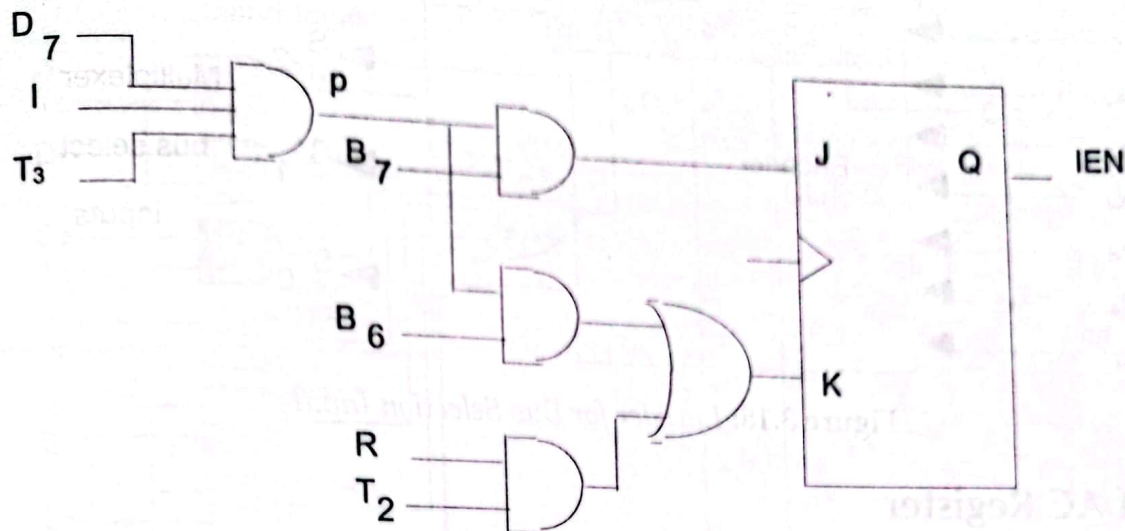


Figure 3.17: Control Input for IEN

Control of Common Bus

16-bit common bus is controlled by three selection inputs S_2 , S_1 and S_0 . The decimal number associated with each component of the bus determines the equivalent binary number that must be applied to the selection inputs to select the registers. This is described by the truth table given below:

Table 3.7: Encoder for Bus Selection Circuit

Inputs							Outputs			Register selected for bus
x_1	x_2	x_3	x_4	x_5	x_6	x_7	S_2	S_1	S_0	
0	0	0	0	0	0	0	0	0	0	None
1	0	0	0	0	0	0	0	0	1	AR
0	1	0	0	0	0	0	0	1	0	PC
0	0	1	0	0	0	0	0	1	1	DR
0	0	0	1	0	0	0	1	0	0	AC
0	0	0	0	1	0	0	1	0	1	IR
0	0	0	0	0	1	0	1	1	0	TR
0	0	0	0	0	0	1	2	1	1	Memory

To find the logic that makes $x_1=1$ we scan and extract all the statements that use AR as source.

D_4T_4 : $PC \leftarrow AR$

D_5T_5 : $PC \leftarrow AR$

$\Rightarrow$

$$x_1 = D_4T_4 + D_5T_5$$

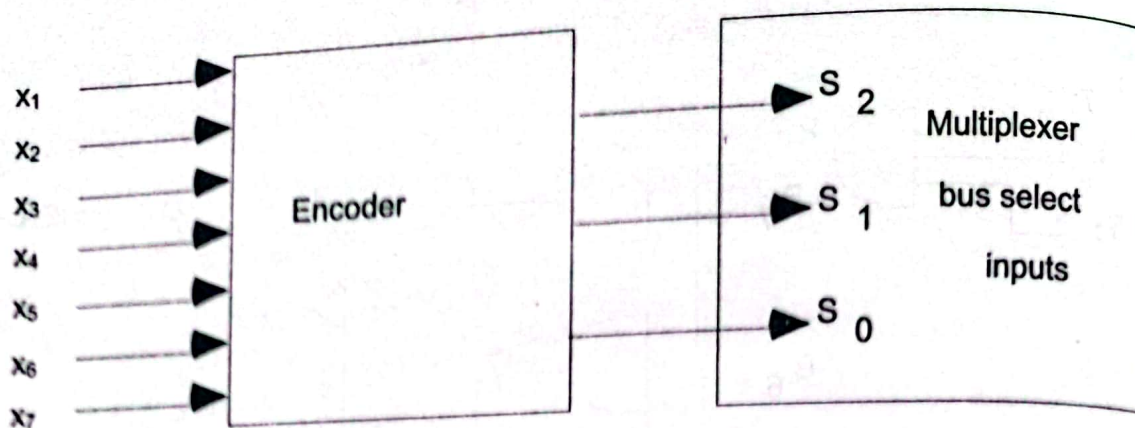


Figure 3.18: Encoder for Bus Selection Inputs.

Control of AC Register

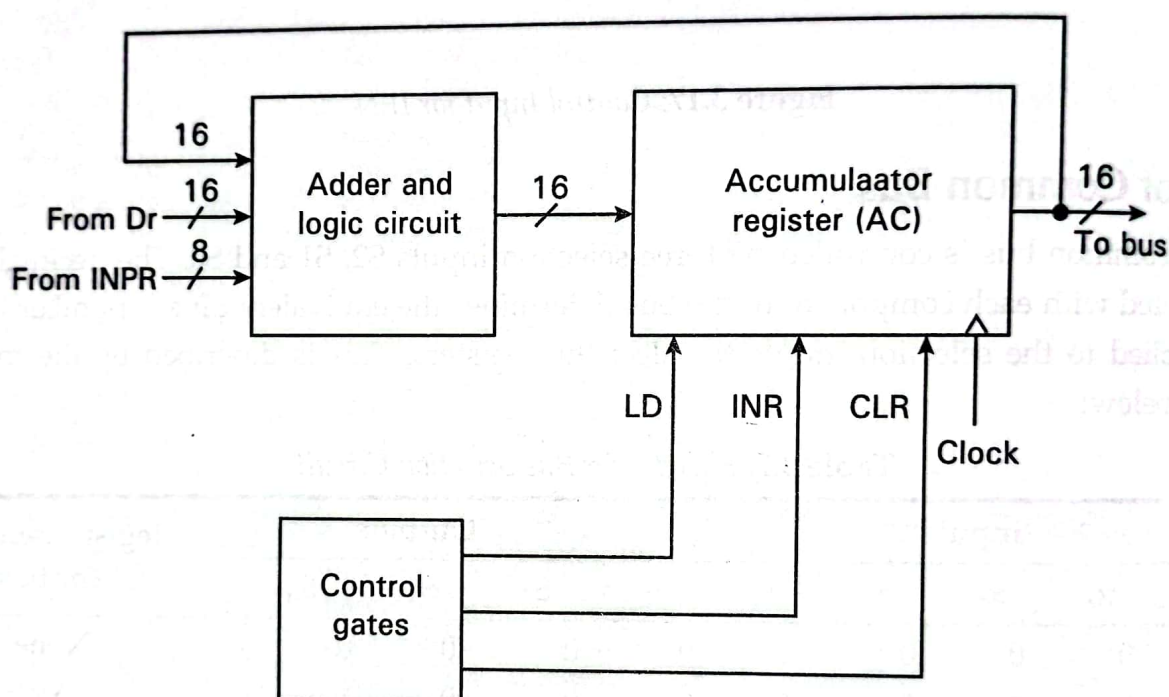


Figure 3.19: Circuits Associated with AC.

All the statements that change the content of AC

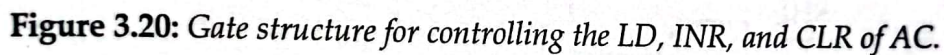
D_0T_5 :	$AC \leftarrow AC \wedge DR$	AND with DR
D_1T_5 :	$AC \leftarrow AC + DR$	Add with DR
D_2T_5 :	$AC \leftarrow DR$	Transfer from DR
pB_{11} :	$AC(0-7) \leftarrow INPR$	Transfer from INPR
rB_9 :	$AC \leftarrow AC'$	Complement
rB_7 :	$AC \leftarrow shr AC, AC(15) \leftarrow E$	Shift right
rB_6 :	$AC \leftarrow shl AC, AC(0) \leftarrow E$	Shift left
rB_{11} :	$AC \leftarrow 0$	Clear
rB_5 :	$AC \leftarrow AC + 1$	Increment

=>

$$LD(AC) = D_0T_5 + D_1T_5 + D_2T_5 + pB_{11} + rB_9 + rB_7 + rB_6$$

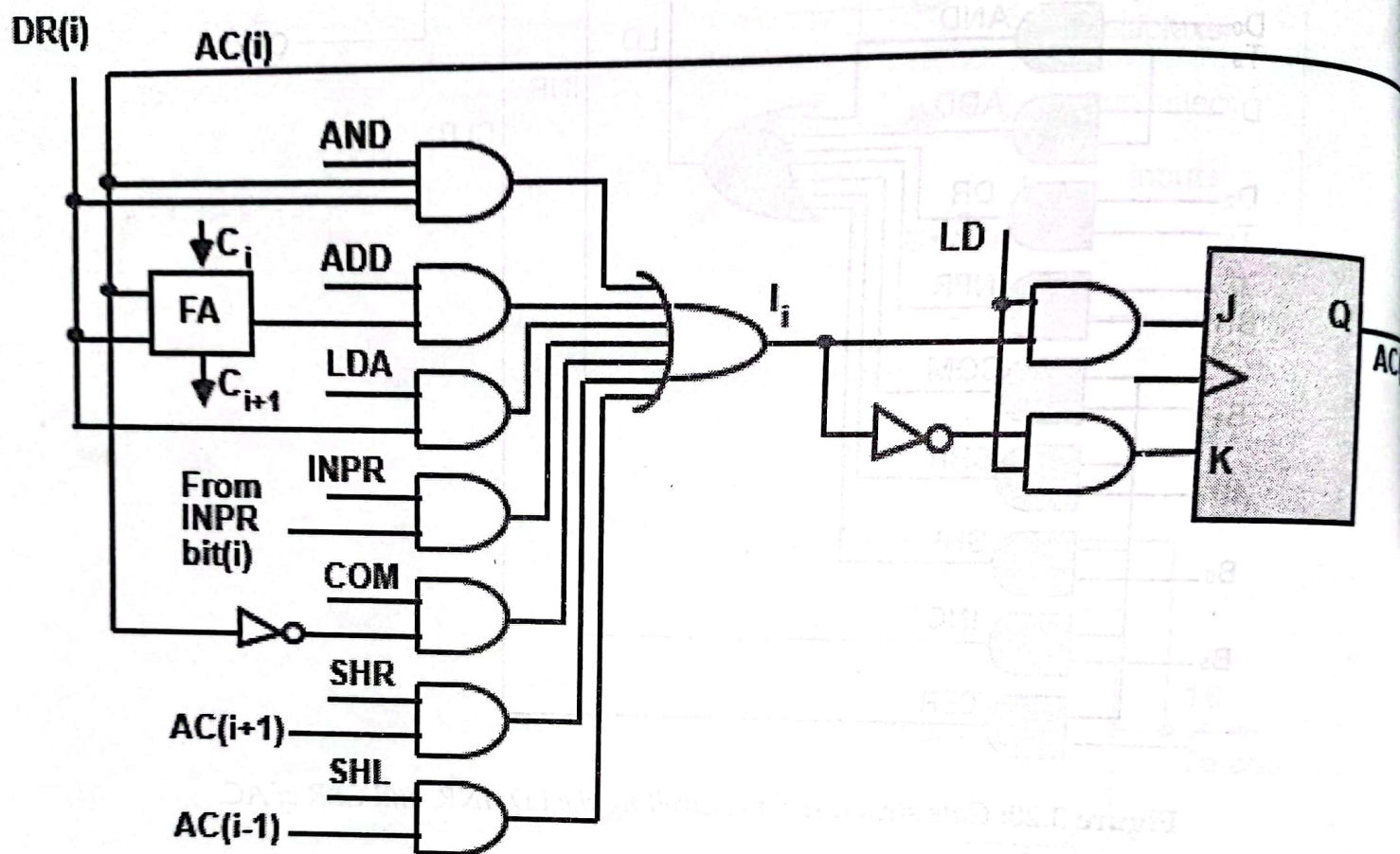
$$CLR(AC) = rB_{11}$$

$$INR(AC) = rB_5$$



The AND operation is achieved by ANDing AC(i) with the corresponding bit in the data register DR (i). The ADD operation is obtained using a binary adder.

Figure 3.21: One stage of adder and logic circuit



The adder and logic circuit can be used to add two numbers. The input to the adder is the number to be added, and the input to the logic circuit is the number to be added to. The output of the adder is the sum, and the output of the logic circuit is the carry. The carry is used to add the next bit of the number to be added.

One stage of the adder and logic circuit is shown in Figure 3.21. The adder and logic circuit are connected to the output of the adder and the output of the logic circuit. The output of the adder is the sum, and the output of the logic circuit is the carry. The carry is used to add the next bit of the number to be added.